SENTINEL

An AI-assisted application security scanner that explains what it finds and fixes it itself

Document	Sentinel, Product Requirements
Version	1.0
Status	Draft
Date	July 29, 2026
Author	Satyam Pandey
Repository	github.com/SatyamPandey07/Application-Security-Scanner
Live Demo	sentinel-application.netlify.app
License	MIT

What This Is, and Why It Exists

Sentinel scans source code and running web applications for security problems, the same way a security team's checklist would, except it runs five different scanning tools at once and doesn't get tired doing it. It looks for insecure code patterns, live vulnerabilities in a running app, whether one user can peek at another user's data, outdated packages with known CVEs, and secrets like API keys accidentally left in the code.

What separates it from a typical scanner is what happens after something is found. Most scanners hand back a long list of alerts and leave the sorting out to a human, and a lot of that list turns out to be noise. Sentinel runs every finding through Claude, which throws out the low-confidence noise, explains the real ones in plain language with a concrete exploit scenario, and writes an actual code fix. If the fix looks right, one click opens a pull request on GitHub with that fix already applied on a new branch.

The Problem It's Solving

Security scanning tools already exist, plenty of them. Semgrep catches bad code patterns. OWASP ZAP finds live vulnerabilities in a running app. Dependency checkers flag outdated packages. The tools aren't the hard part. The hard part is what happens after the scan finishes: a security engineer staring at two hundred findings, most of which are false positives or duplicates, trying to figure out which five actually matter today, then writing the fix for each one by hand.

> *Sentinel's bet is that the scanning half of this problem is basically solved already. The unsolved half is triage and remediation, and that's the half it's built around.*

Who Actually Uses This

- **Security engineers.** Run scans against applications the team owns, review the AI's findings and confidence filtering, and decide which fixes actually get merged.
- **Developers on the receiving end.** See a pull request land with a security fix already written and explained, instead of a vague ticket telling them to go figure it out.

- **Admins.** Manage who's allowed to do what through role-based access control, since scanning tools that touch other systems need real access boundaries.
- **Compliance and audit teams.** Pull CSV reports mapped directly to SOC 2, PCI DSS, and OWASP ASVS requirements instead of translating raw findings by hand.

The Rule Everything Else Depends On

A tool that scans live web applications and pokes at authorization boundaries is, by nature, capable of being pointed at something the user doesn't actually own. Sentinel treats that as the single most important constraint in the whole system, not an afterthought.

- Every scan request has to include an explicit authorized: true flag before anything runs. There's no default-on, silent-scan path.
- That confirmation is written to a dedicated consent_log table in PostgreSQL, so there's a durable record of who authorized what, and when.
- The self-hosting checklist calls this out as the first item to verify before going to production, ahead of encryption keys or rate limits.

How the System Is Put Together

Five services cooperate to make one scan happen: a frontend someone actually looks at, a backend that handles requests and permissions, a queue that holds pending work, a worker that runs the actual scanning tools, and the AI and scoring layer that turns raw findings into something a person can act on.

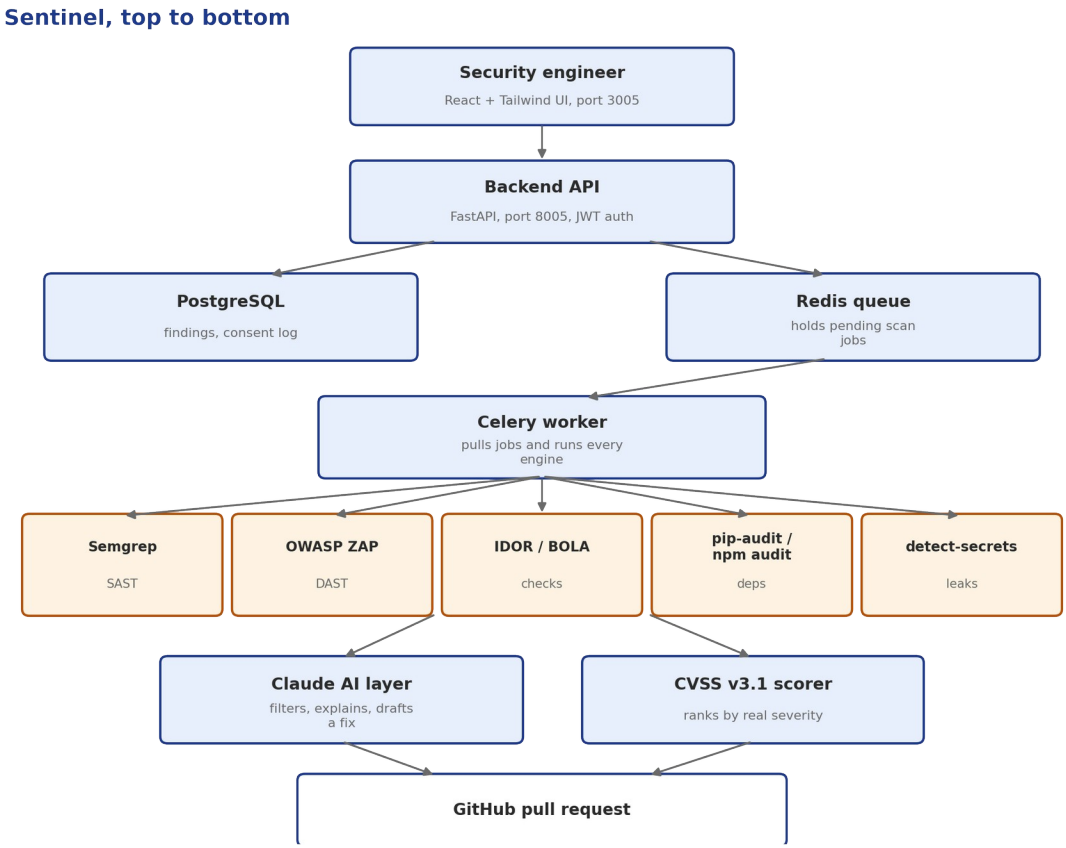

Figure 1. Request in, pull request out.

Piece	Job	Built with
Frontend	Where a user submits a target and reviews results	React 18, Vite, Tailwind CSS
Backend API	Handles auth, permissions, and every REST request	FastAPI, Python 3.12
PostgreSQL	Stores findings, scan history, and the consent log	PostgreSQL 16, SQLAlchemy, Alembic
Redis queue	Holds scan jobs until a worker is free to run them	Redis 7
Celery worker	Pulls jobs off the queue and runs every scan engine	Celery 5.3

The five scan engines all run from inside that Celery worker, each one aimed at a different class of problem: Semgrep reads source code for unsafe patterns, OWASP ZAP pokes at a running application the way an attacker would, a custom IDOR and BOLA checker tests whether one logged-in user can reach another user's data, pip-audit and npm audit check dependencies against known CVE databases, and detect-secrets scans for credentials and keys that shouldn't be sitting in the code.

What Happens to a Finding After It's Found

Running five scanners is the easy part. What happens next is the part that actually saves someone time.

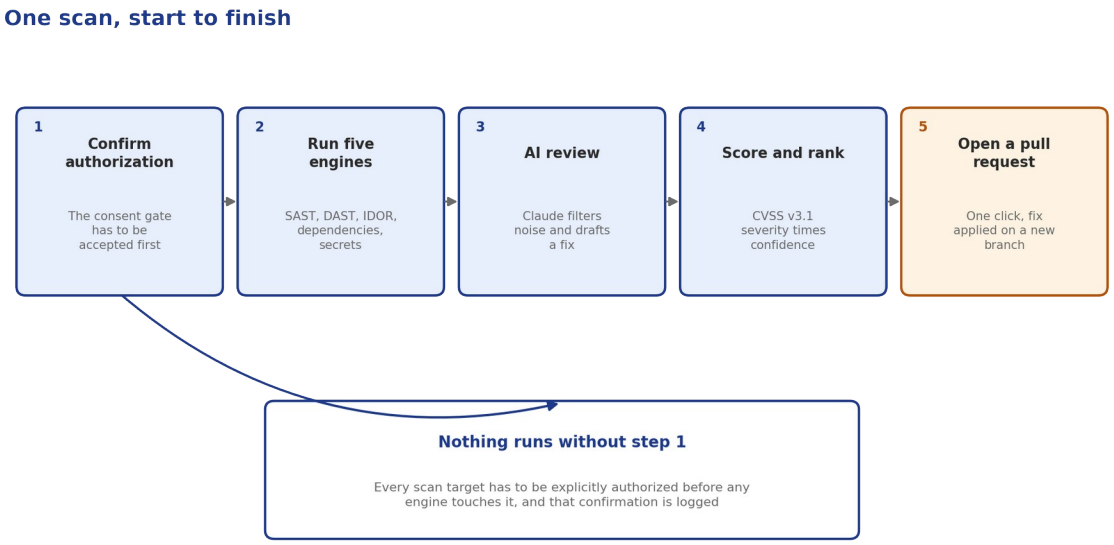

Figure 2. From submitted target to a fix already sitting in a pull request.

1. A scan target is submitted, along with the required authorization confirmation.
2. The job goes into the Redis queue and a Celery worker picks it up and runs all five engines.
3. Every finding goes through Claude, which reads the surrounding source code, throws out low-confidence noise, writes a plain-language explanation with a realistic exploit scenario, and drafts an actual patch.
4. Each surviving finding gets a CVSS v3.1 base score, multiplied against the AI's own confidence rating, so the riskiest, most-certain issues float to the top instead of getting buried.
5. A person reviews the finding and the suggested patch on the dashboard.
6. One click opens a new branch and a pull request on GitHub with that patch already applied, ready for normal code review.

Everything the Product Actually Does

Finding problems

Engine	What it catches
Static code analysis (SAST)	Insecure coding patterns and known bug classes in source code, using Semgrep's security-audit ruleset
Dynamic testing (DAST)	Live vulnerabilities in a running application, using OWASP ZAP
Authorization and IDOR testing	Whether a logged-in user can view or modify data that belongs to someone else
Dependency auditing	Project packages with known CVEs, checked with pip-audit and npm audit
Secret leak detection	API keys, database credentials, and tokens accidentally committed into the code

Making sense of what was found

- Claude reviews every finding next to the actual surrounding code, not just the isolated line that triggered it.
- Each issue gets explained in plain language along with a concrete, step-by-step exploit scenario, not just a rule ID.
- Low-confidence alerts are filtered out automatically, so a reviewer's attention goes to things that are actually likely to be real.
- A CVSS v3.1 score is calculated for every surviving finding and combined with AI confidence to produce the priority ranking shown on the dashboard.

Closing the loop

- One click opens a branch and a pull request on GitHub with the suggested fix already applied, ready for a developer to review and merge.
- Findings map directly to SOC 2, PCI DSS v4.0, and OWASP ASVS v4.0 requirements, and can be exported as a CSV for an audit.
- Scan history is tracked over time, so a team can see whether their overall risk is actually trending down, not just look at one scan in isolation.

Security Model, In the Product Itself

A tool this powerful needs its own access controls taken just as seriously as the vulnerabilities it's hunting for.

- Every user authenticates with a JWT, checked by dedicated security middleware on the backend for every request.
- Role-based access control separates ordinary users from admins, who alone can manage roles and platform-wide settings.
- Cloned repositories are scanned inside sandboxed temporary directories, created fresh for each job and unconditionally destroyed afterward, even if the scan fails partway through.
- Scan inputs are sanitized before anything runs: path traversal sequences, shell metacharacters, and other malicious arguments are rejected up front.

- Worker containers run with restricted network access specifically to stop a scanned repository from reaching internal cloud metadata endpoints during a scan.

Getting It Running

Fastest path: Docker Compose

- Clone the repository, copy .env.example to .env, and set ANTHROPIC_API_KEY and SECRET_KEY.
- Run docker-compose -f docker-compose.prod.yml up -d.
- The frontend comes up on port 3005, the FastAPI backend on port 8005 (with interactive docs at /docs), and the OWASP ZAP daemon on port 8080.

Running it by hand for development

- Backend: create a virtual environment, pip install -r requirements.txt, run alembic upgrade head, seed the database, then start uvicorn app.main:app on port 8005.
- Frontend: npm install, then npm run dev with the port set to 3005.
- Tests: cd backend, then pytest runs the full backend test suite.

Before This Goes Anywhere Near Production

The project's own deployment guide includes a self-hosting checklist, and it's worth carrying over here directly rather than paraphrasing it into something vaguer.

- ☐ The mandatory authorized: true consent gate on POST /scans is still in place and logging to consent_log.
- ☐ SECRET_KEY has been changed from its default to a strong, random 64-character value.
- ☐ Celery worker containers run with restricted network privileges, so a scanned repository can't reach internal cloud metadata endpoints.
- ☐ Rate limiting is active on /scans (10 requests per minute per user).
- ☐ The OWASP ZAP container's port 8080 is only reachable from the internal API and worker network, never exposed publicly.
- ☐ Daily automated backups are configured for the postgres_data volume.

Environment Variables

Variable	What it's for	Required in production
DATABASE_URL	PostgreSQL connection string	Yes
REDIS_URL	Redis task queue broker URL	Yes
ZAP_API_URL	OWASP ZAP REST API daemon URL	Yes
SECRET_KEY	JWT signing secret	Yes
AI_VALIDATION_ENABLED	Turns the AI false-positive filtering layer on or off	No
ANTHROPIC_API_	Needed for Claude-powered remediation to work at all	Optional, but

Variable	What it's for	Required in production
KEY		effectively required for the AI features

What's Not Here Yet

- No support for scanning mobile app binaries or infrastructure-as-code templates, the five engines are focused on web application code and running web targets.
- No built-in ticketing integration such as Jira. Findings currently live in the dashboard and flow out through GitHub pull requests or CSV export.
- No continuous, always-on monitoring mode. A scan is a discrete, explicitly authorized job, not a background watcher.
- The AI remediation layer currently runs on Claude 3 Haiku specifically, chosen for cost and speed rather than the largest available model.

What Success Looks Like

Signal	What we'd want to see	How we'd check
Findings are worth reading	A high share of AI-surfaced findings turn out to be real, not noise	Manual review of accepted versus dismissed findings over time
Fixes actually get merged	A meaningful share of auto-generated pull requests get merged with little or no edits	GitHub PR merge rate for Sentinel-opened branches
Time to fix drops	Less time between a scan finishing and the underlying issue being resolved	Timestamp comparison between finding creation and PR merge
Nobody scans without permission	Every scan in the system has a matching, valid consent_log entry	Database audit comparing scans to consent records
Compliance reporting is actually used	Teams regularly export CSV reports ahead of audits rather than building their own	Export event tracking, if usage analytics are added

Where This Could Go Wrong

Risk	Why it matters	What helps
The consent gate gets bypassed or misconfigured	Turns a security tool into something that could scan systems without permission, a serious legal and ethical problem	Treat the authorized: true check and consent_log write as non-negotiable, and cover it with its own dedicated tests
A scanned repository escapes its sandbox	Malicious code in a scanned repo could reach internal infrastructure	Isolated temporary directories per job, unconditional cleanup, and

Risk	Why it matters	What helps
	during analysis	restricted network access for worker containers
AI-suggested fixes get merged without real review	A subtly wrong automated fix could introduce a new bug or even a new vulnerability	Every fix lands as a normal pull request for human review, not an automatic merge
Too many false positives survive AI filtering	Erodes trust fast, people start ignoring findings the same way they ignore any noisy alert system	Keep tuning the confidence filtering, and track dismissed-versus-real findings as a real product metric
Dependence on a single AI provider and model	An outage or pricing change to Claude 3 Haiku directly affects the remediation layer	Keep AI_VALIDATION_ENABLED as a real off switch so scanning still works with that layer disabled

Open Questions

- Should the remediation layer support choosing a different Claude model, or another provider entirely, rather than being fixed to Claude 3 Haiku?
- What does a good escalation path look like if a scan finds something urgent, like an actively exploitable issue, outside of the normal dashboard review flow?
- Is there a case for a lighter continuous monitoring mode, still gated by consent, for teams that want more than a one-off scan?
- Would native ticketing integrations (Jira, Linear) meaningfully reduce friction for teams that don't run everything through GitHub pull requests?

Quick Reference

Item	Value
Frontend	React 18, Vite, Tailwind CSS, port 3005
Backend API	FastAPI, Python 3.12, port 8005 (docs at /docs)
Database	PostgreSQL 16, SQLAlchemy 2.0, Alembic
Task queue	Redis 7 with Celery 5.3 workers
OWASP ZAP daemon	Port 8080, internal network only in production
AI remediation	Anthropic Claude API, claude-3-haiku-20240307
Start in production	docker-compose -f docker-compose.prod.yml up -d
Run backend tests	cd backend, then pytest
Live demo	sentinel-application.netlify.app